

MODULE 6

WHERE

1. Show all products in the Electronics category.
2. Show all customers who live in Chennai or Bengaluru and signed up after 1 January 2025.
3. Show all products that are not in Electronics and have total worth above 200000, or any product with price below 500 and stock of 300 or more.

ORDER BY

1. Show all products sorted by price from lowest to highest.
2. Show all customers sorted by city A–Z, then by name Z–A within each city.
3. Show `product_name` and total worth (price $\times$ stock), sorted by total worth from highest to lowest, using the alias in `ORDER BY`.

LIMIT

1. Show the first 5 customers.
2. Show the 3 most expensive products.
3. Show the products ranked 4th to 6th by price (use `LIMIT` with `OFFSET`).

DISTINCT

1. Show all unique cities from `customers`.
2. Show all unique product categories.
3. Show all unique combinations of `customer_id` and `status` from `orders`, sorted by `customer_id`.

COUNT

1. Count the total number of customers.
2. Count how many orders have the status 'delivered'.
3. Count how many distinct customers have placed at least one order.

SUM

1. Find the total quantity of items across all of `order_items`.
2. Find the total stock of all Electronics products.
3. Find the total worth (price $\times$ stock) of each category, with each category shown as one row.

AVG

1. Find the average price of all products.
2. Find the average price of products in the Accessories category.
3. Show each category with its average price, but only categories whose average price is above 5000.

MIN

1. Find the lowest product price.
2. Find the earliest order date.

3. Show each city with the earliest signup date of its customers.

MAX

1. Find the highest product price.
 2. Find the latest order date among orders that are not cancelled.
 3. Show each category with its most expensive product's price, sorted from highest to lowest.
-

MODULE 7

INNER JOIN

1. Show each order's `order_id`, order date and the customer's name.
2. Show `order_id`, product name and quantity by joining `order_items` and `products`.
3. Show customer name, `order_id`, product name and quantity for all delivered orders (join four tables).

LEFT JOIN

1. Show all customers with their `order_ids`, including customers with no orders.
2. Show only the customers who have never placed an order.
3. Show every product with the total quantity sold, showing 0 for products never sold (use `IFNULL` or `COALESCE`).

RIGHT JOIN

1. Show all orders and the customer name using a `RIGHT JOIN` (customers on the left, orders on the right).
2. Show all products with the `order_id` they appear in, using `RIGHT JOIN` from `order_items` to `products`.
3. Rewrite your earlier question "customers who have never placed an order" using a `RIGHT JOIN`, and confirm it gives the same result as your `LEFT JOIN` version.

FULL JOIN

MySQL has no `FULL JOIN`, so you build it with `LEFT JOIN ... UNION ... RIGHT JOIN`.

1. Show all customers and all orders together, matched where possible.
2. Show all products and all `order_items` together, matched where possible.
3. Show only the rows that do not match on either side (customers with no orders and orders with no customer), using the full-join pattern.

Multi-table Queries

1. Show the total number of orders per city (join `customers` and `orders`).
2. Show the total revenue per customer (quantity $\times$ price) across all non-cancelled orders.
3. Show the top 3 best-selling products by total quantity sold, with their category and total revenue.

Subqueries

1. Show products whose price is above the average price.
2. Show customers who have placed at least one order (use **IN**).
3. Show customers who have never ordered any Electronics product (use **NOT IN** or **NOT EXISTS**).

Nested Queries

1. Show the product with the highest price using a nested query (no **LIMIT**).
2. Show the names of customers who ordered the most expensive product.
3. Show customers whose total number of orders is above the average number of orders per customer.

Common Table Expressions (CTEs)

1. Write a CTE that selects delivered orders, then count them.
2. Write a CTE that calculates total revenue per order, then show only orders with revenue above 10000.
3. Use two CTEs: one for revenue per customer and one for the average of those revenues. Show customers whose revenue is above that average.

Complex Filtering

1. Show orders placed in 2026 whose status is either 'shipped' or 'pending'.
2. Show customers from Kochi or Chennai who have placed more than 1 order (use **HAVING**).
3. Show products that were ordered by at least 2 different customers and have stock below 100.

Window Functions: Ranking

1. Rank all products by price from highest to lowest using **RANK()**.
2. Give each customer a row number ordered by signup date using **ROW_NUMBER()**.
3. Show the top 2 most expensive products in each category using **DENSE_RANK()** inside a CTE.

Window Functions: Partitioning

1. Show every product with the average price of its category next to it (**AVG()** **OVER** (**PARTITION BY** . . .)).
2. Show every order with the number of orders its customer has placed in total.
3. Show every product with its price, the category maximum price, and how far below the maximum it is.

Window Functions: Running Totals

1. Show orders with a running count of orders by order date.
2. Show each order's total quantity and a running total of quantity ordered by **order_date**.
3. Show monthly revenue (grouped by month) with a running total of revenue across months.

Indexes & Query Optimization

1. Create an index on `customers(city)` and check it with `SHOW INDEX FROM customers;`.
2. Run `EXPLAIN` on `SELECT * FROM orders WHERE customer_id = 3;` before and after creating an index on `orders(customer_id)`, and compare the `type` and `rows` values.
3. Create a composite index on `orders(status, order_date)`. Then use `EXPLAIN` to compare a query that filters on both columns with one that filters only on `order_date`, and explain why the results differ.